

Load Balancer with Multi-Backend Messaging Application

CSD Course — Lab Assignment 5

Student Name	Rahul Raj
Roll Number	12341680
Date	24 August 2026
Public URL	http://10.1.75.53:3269 (served through the load balancer)
Repository	Lab-Task/5.Load-Balancer (git, tagged per phase)

1. Assigned Systems

Role	Alias	SSH access	Container IP	Service port	External port
Load balancer + shared state	stu78_sys1	ssh -p 2269 student@10.1.75.53	172.17.0.70	LB 3000 · state 5269	3269 → LB
Backend A	stu78_sys2	ssh -p 2270 student@10.1.75.53	172.17.0.71	app 3000	3270 (direct baseline)
Backend B	stu78_sys3	ssh -p 2271 student@10.1.75.53	172.17.0.72	app 3000	3271
Backend C	stu78_sys4	ssh -p 2272 student@10.1.75.53	172.17.0.73	app 3000	3272

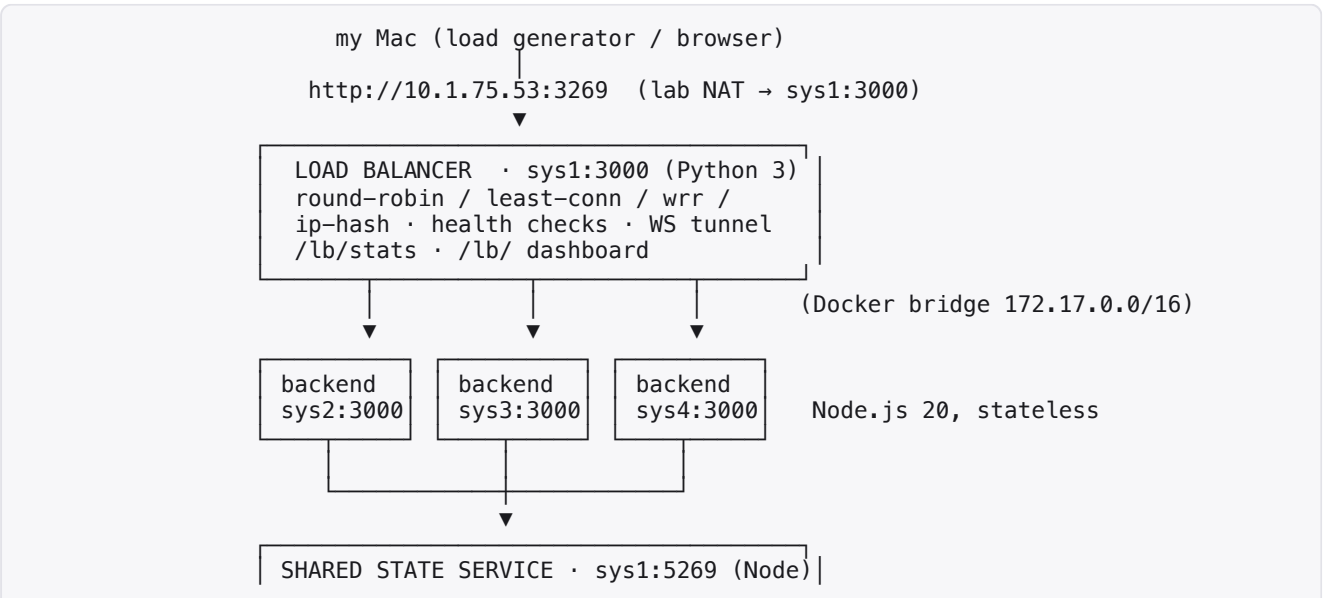
The four systems are Ubuntu 24.04 containers on one lab host, each limited by cgroup to **exactly 1 CPU core** (`cpu.max = 100000/100000`) — a fact that shapes every measurement in this report. The lab NAT forwards only the first assigned external port of each system, mapping it to container port 3000 (discovered empirically in Phase 0, `evidence/00_env_audit.md`). The load generator runs on my local macOS machine.

2. Objective & Assignment Statement

Make a Load Balancer on Sys1. Deploy back-end on Sys2, Sys3 and Sys4 — use the messaging app back-end developed in the previous assignment. Make a Load Generator on your local system sending load to your Load Balancer. Measure metrics with the LB using only Sys2, then using all three back-ends. Compare the results and add a comparison table. Integration: the previous assignment's messaging URL must work through the new load-balanced back-end.

All requirements are met; the comparison table is in §11 and the integration proof in §13.

3. Architecture



```
users · sessions · rooms · message log  
+ WebSocket firehose → all backends
```

Request lifecycle. A client request hits the LB, which picks a healthy backend (default round-robin), adds `X-Forwarded-For` / `X-Real-IP`, and proxies over a pooled keep-alive connection. The backend validates the session (cached 30 s, source of truth in the state service), performs the operation against the state service, and answers with an `X-Backend-Id` header identifying itself — visible as a badge in the UI and recorded by the load generator. New messages are appended to the state service's log, which broadcasts them on a WebSocket firehose; every backend forwards the event to its own connected clients. This is what makes three stateless backends behave as one application: a user connected to sys4 instantly receives a message that entered through sys2 (proof: [evidence/03_cross_instance_delivery.txt](#)).

WebSocket connections from browsers are detected at the LB via the `Upgrade` header, pinned to one backend by IP-hash, and tunnelled byte-for-byte.

4. Messaging Application (Assignment-4 backend, rebuilt clean)

The backend is the Assignment-4 group-chat application ("ChatFat") — same security model and wire format, rebuilt to be horizontally scalable and to run on the lab's Node runtime, with a cleaner single-page UI.

Features. Register / login, room list, create room, plaintext rooms, end-to-end encrypted ("locked") rooms, live delivery over WebSocket with automatic reconnect and a polling fallback, connection-state indicator, and a badge showing which backend served you.

Auth. `script` password hashing (per-user random 16-byte salt, `crypto.script`, constant-time compare), `HttpOnly SameSite=Strict` session cookies (48-hex random token, 12 h expiry), and per-IP rate limiting of **failed** sign-ins (10/min — Assignment-4 semantics). Behind the LB the limiter keys on `X-Real-IP`, not the socket address. Assignment 4's later `better-auth` integration requires Node ≥ 22 and could not boot on the lab's Node 18 — restoring the original zero-dependency `script` design fixed that incompatibility (bug list in §14).

End-to-end encryption — preserved exactly from Assignment 4:

```
salt      = SHA-256("ChatFat-room-v1|" + roomId)           (deterministic)  
key K     = PBKDF2-HMAC-SHA-256(passphrase, salt, 250 000 iterations) → 256 bit  
per msg   : AES-256-GCM, fresh 96-bit IV, random 128-bit nonce n  
AAD       = roomId || "|" || b64(n) || "|" || keyEpoch    (no cross-room replay)  
envelope  = { alg:"A256GCM", kid:epoch, n, iv, ct, aadv:1 }  
verifier  = HMAC-SHA-256(K, "ChatFat-room-verify|roomId|epoch") (join check)
```

Key derivation happens **in the browser** (WebCrypto); the passphrase never leaves it. The server validates only envelope shape and size (IV exactly 12 bytes, ciphertext ≤ 12 KiB) and stores ciphertext it cannot read — proven by inspecting the store ([evidence/02_ciphertext_proof.txt](#), screenshot 07). Locking is forward-only (no downgrade attack); re-locking bumps the key epoch.

API surface.

Endpoint	Method	Purpose
/api/register, /api/login, /api/logout	POST	script auth, session cookie
/api/me	GET	session probe
/api/rooms	GET/POST	directory / create
/api/rooms/:id/lock	POST	lock with epoch + verifier
/api/messages?room=&since=	GET	history (seq-based)
/api/messages	POST	send (plaintext or E2E envelope)
/ws	WS	live delivery (receive), pinned by LB
/health, /metrics, /whoami	GET	LB integration & observability

Sends travel over HTTP POST (never blind-retried by the LB — no duplicate messages); receives travel over the pinned WebSocket.

Tests. `app/tests/smoke.js` boots a real state service plus two backend instances and asserts 24 conditions: auth paths, sessions created on one instance validating on the other, E2E encrypt→store→fetch→decrypt round-trip against an independent reimplement of the crypto, ciphertext-only storage on disk, malformed-envelope rejection, and cross-instance WebSocket delivery. All pass (`evidence/02_smoke_all_pass.log`).

5. Load Balancer Design (sys1)

`lb/loadbalancer.py` — ~580 lines of pure Python 3 standard library; full source in §6.

- **Algorithms** (config-switchable): `round_robin` (default), `least_connections`, `weighted_round_robin` (nginx-style smooth WRR), `ip_hash` (sticky).
- **Active health checks**: background thread GETs each backend's `/health` every 5 s; ejection after 3 consecutive failures, re-admission after 2 successes (hysteresis). Timeout is 4 s because a saturated 1-core backend can take seconds to answer — the original 2 s timeout caused spurious ejections under load (§14).
- **Passive health checks**: a connect failure during proxying ejects the backend immediately and retries the next healthy one. Retries happen only when zero bytes were sent upstream, so a POST can never be duplicated.
- **WebSockets**: `Upgrade` requests are pinned via IP-hash and tunnelled raw in both directions; the LB stays out of the frame protocol.
- **Proxy correctness**: adds `X-Forwarded-For` (appending), `X-Real-IP`, `X-Forwarded-Proto`; preserves `Host`; strips hop-by-hop headers.
- **Performance**: thread-per-connection with client keep-alive and an upstream connection pool per backend (idle sockets are liveness-checked with `MSG_PEEK` before reuse).
- **Observability**: per-request CSV access log (timestamp, client, method, path, chosen backend, upstream ms, status, bytes); `/lb/stats` JSON (per-backend counts, in-flight, errors, health, p50/p95/p99); an auto-refreshing HTML dashboard at `/lb/` (screenshot 04). Config lives in `lb/lb.conf.json`; `POST /lb/reload` (or `SIGHUP`) re-reads it without dropping connections — switching the B and C experiment pools is a config change, not a code change.

6. Load Balancer — Full Source Code

```

1  #!/usr/bin/env python3
2  """
3  loadbalancer.py – HTTP/WebSocket reverse-proxy load balancer.
4
5  Assignment 5, CSD course. Runs on sys1 and spreads traffic from
6  http://10.1.75.53:3269 across the messaging-app backends on sys2/sys3/sys4.
7
8  Pure Python 3 standard library – no pip, no root, no external processes.
9
10 Features
11 -----
12 * Four selection algorithms, switchable in lb.conf.json:
13     round_robin          each request goes to the next healthy backend
14     least_connections    the backend with the fewest in-flight requests
15     weighted_round_robin smooth WRR honouring per-backend "weight"
16     ip_hash              same client IP -> same backend (sticky)
17 * Active health checks: a background thread polls every backend's /health;
18   a backend is ejected after `fail_threshold` consecutive misses and
19   re-admitted after `rise_threshold` consecutive successes.
20 * Passive health checks: a connect failure during proxying ejects the backend
21   immediately and the request is retried on the next healthy backend
22   (only when nothing was sent upstream yet – safe for any method).
23 * WebSocket support: an Upgrade request is pinned to one backend and the
24   connection is pumped byte-for-byte in both directions until either side
25   closes (the LB stays out of the framing).
26 * Proxy headers: adds X-Forwarded-For, X-Real-IP, X-Forwarded-Proto,
27   preserves Host, strips hop-by-hop headers.
28 * Upstream connection pooling + client keep-alive so the LB itself is not
29   the bottleneck.
30 * Observability: per-request CSV access log, JSON stats at /lb/stats and a
31   small auto-refreshing HTML dashboard at /lb/ (both served by the LB
32   itself on the public port – only port 3269 is NAT-forwarded in this lab).
33   /lb/reload re-reads lb.conf.json without dropping connections.
34
35 Usage: python3 loadbalancer.py [path/to/lb.conf.json]
36 """
37
38 import json
39 import os
40 import signal
41 import socket
42 import sys
43 import threading
44 import time
45 from collections import deque
46
47 # ----- configuration -----
48
49 CONF_PATH = sys.argv[1] if len(sys.argv) > 1 else os.path.join(
50     os.path.dirname(os.path.abspath(__file__)), "lb.conf.json")
51
52 DEFAULTS = {
53     "listen_host": "0.0.0.0",
54     "listen_port": 3269,
55     "algorithm": "round_robin",
56     "backends": [], # [{"id": "sys2", "host": "...", "port": 3270, "weight":
57 1}]
58     "health_interval_s": 3,
59     "health_timeout_s": 2,
60     "fail_threshold": 2,
61     "rise_threshold": 2,
62     "connect_timeout_s": 3,
63     "upstream_timeout_s": 30,
64     "access_log": "logs/lb_access.csv",
65 }
66
67
68 class Backend:
69     """One upstream server plus its live health/traffic bookkeeping."""
70
71     def __init__(self, cfg):
72         self.id = cfg["id"]
73         self.host = cfg["host"]

```

```

74     self.port = int(cfg["port"])
75     self.weight = int(cfg.get("weight", 1))
76     self.healthy = True # optimistic until the first check
77     self.consec_fail = 0
78     self.consec_ok = 0
79     self.in_flight = 0
80     self.requests = 0
81     self.errors = 0
82     self.latencies = deque(maxlen=5000) # upstream ms, newest last
83     self.current_weight = 0 # smooth-WRR state
84     self.pool = deque() # idle keep-alive sockets to this backend
85     self.lock = threading.Lock()
86
87     def addr(self):
88         return (self.host, self.port)
89
90     def snapshot(self):
91         lat = sorted(self.latencies)
92         pct = lambda p: round(lat[min(len(lat) - 1, int(p / 100 * len(lat)))], 1) if lat else
93 0
94         return {
95             "id": self.id, "host": self.host, "port": self.port,
96             "weight": self.weight, "healthy": self.healthy,
97             "in_flight": self.in_flight, "requests": self.requests,
98             "errors": self.errors,
99             "latency_ms": {"p50": pct(50), "p95": pct(95), "p99": pct(99)},
100         }
101
102
103 class LB:
104     def __init__(self):
105         self.cfg = dict(DEFAULTS)
106         self.backends = []
107         self.rr_index = 0
108         self.lock = threading.Lock()
109         self.started = time.time()
110         self.total_requests = 0
111         self.total_errors = 0
112         self.log_lock = threading.Lock()
113         self.log_fh = None
114         self.reload()
115
116     # -- config -----
117     def reload(self):
118         with open(CONF_PATH) as fh:
119             file_cfg = json.load(fh)
120             cfg = dict(DEFAULTS)
121             cfg.update(file_cfg)
122         with self.lock:
123             old = {b.id: b for b in self.backends}
124             fresh = []
125             for bc in cfg["backends"]:
126                 if bc["id"] in old: # keep live stats across reloads
127                     b = old[bc["id"]]
128                     b.host, b.port = bc["host"], int(bc["port"])
129                     b.weight = int(bc.get("weight", 1))
130                     fresh.append(b)
131                 else:
132                     fresh.append(Backend(bc))
133             self.backends = fresh
134             self.cfg = cfg
135             log_path = cfg["access_log"]
136             os.makedirs(os.path.dirname(log_path) or ".", exist_ok=True)
137             new_log = not os.path.exists(log_path)
138             if self.log_fh:
139                 self.log_fh.close()
140             self.log_fh = open(log_path, "a", buffering=1)
141             if new_log:
142                 self.log_fh.write("ts,client,method,path,backend,upstream_ms,status,bytes\n")
143             print(f"[lb] config loaded: {cfg['algorithm']}, "
144                   f"{len(self.backends)} backend(s): {[b.id for b in self.backends]}")
145
146     # -- backend selection -----
147     def healthy_backends(self):

```

```

148         return [b for b in self.backends if b.healthy]
149
150     def pick(self, client_ip, exclude=()):
151         """Choose a backend for this request using the configured algorithm."""
152         pool = [b for b in self.healthy_backends() if b.id not in exclude]
153         if not pool:
154             return None
155         algo = self.cfg["algorithm"]
156         with self.lock:
157             if algo == "least_connections":
158                 return min(pool, key=lambda b: (b.in_flight, b.requests))
159             if algo == "ip_hash":
160                 return pool[hash(client_ip) % len(pool)]
161             if algo == "weighted_round_robin":
162                 # Smooth WRR (nginx algorithm): pick highest current_weight,
163                 # then subtract the total - spreads weights evenly over time.
164                 total = sum(b.weight for b in pool)
165                 for b in pool:
166                     b.current_weight += b.weight
167                 chosen = max(pool, key=lambda b: b.current_weight)
168                 chosen.current_weight -= total
169                 return chosen
170             # default: round_robin
171             self.rr_index = (self.rr_index + 1) % len(pool)
172             return pool[self.rr_index]
173
174     # -- access log -----
175     def log(self, client, method, path, backend_id, ms, status, nbytes):
176         with self.log_lock:
177             self.log_fh.write(f"{time.time():.3f},{client},{method},{path},"
178                               f"{backend_id},{ms:.1f},{status},{nbytes}\n")
179
180
181     LB_STATE = LB()
182
183     # ----- health checking -----
184
185     def health_loop():
186         """Actively poll every backend's /health; eject/re-admit with hysteresis."""
187         while True:
188             cfg = LB_STATE.cfg
189             for b in list(LB_STATE.backends):
190                 ok = False
191                 try:
192                     with socket.create_connection(b.addr(), timeout=cfg["health_timeout_s"]) as s:
193                         s.sendall(b"GET /health HTTP/1.1\r\nHost: lb-health\r\n"
194                                   b"Connection: close\r\n\r\n")
195                         s.settimeout(cfg["health_timeout_s"])
196                         first = s.recv(1024)
197                         ok = first.startswith(b"HTTP/1.1 200") or first.startswith(b"HTTP/1.0
198 200")
199                 except OSError:
200                     ok = False
201                 if ok:
202                     b.consec_ok += 1
203                     b.consec_fail = 0
204                     if not b.healthy and b.consec_ok >= cfg["rise_threshold"]:
205                         b.healthy = True
206                         print(f"[lb] backend {b.id} re-admitted")
207                 else:
208                     b.consec_fail += 1
209                     b.consec_ok = 0
210                     if b.healthy and b.consec_fail >= cfg["fail_threshold"]:
211                         # Fail-open: never eject the LAST healthy backend. A slow-
212                         # but-alive upstream serving with high latency beats a pool
213                         # with nothing in it (503 for every client).
214                         others = [x for x in LB_STATE.backends
215                                   if x is not b and x.healthy]
216                         if others:
217                             b.healthy = False
218                             print(f"[lb] backend {b.id} EJECTED (health check failed)")
219                         else:
220                             print(f"[lb] backend {b.id} failing health checks but is "
221                                   f"the last one - keeping it (fail-open)")

```

```

222         time.sleep(cfg["health_interval_s"])
223
224
225 def eject_now(b):
226     """Passive check: a live proxy attempt failed – eject immediately,
227     unless this is the last healthy backend (fail-open, as above)."""
228     if b.healthy:
229         others = [x for x in LB_STATE.backends if x is not b and x.healthy]
230         if not others:
231             return
232         b.healthy = False
233         b.consec_fail = LB_STATE.cfg["fail_threshold"]
234         b.consec_ok = 0
235         print(f"[b] backend {b.id} EJECTED (passive: connect/proxy failure)")
236
237
238 # ----- HTTP plumbing -----
239
240 HOP_BY_HOP = {"connection", "keep-alive", "proxy-authenticate",
241               "proxy-authorization", "te", "trailers",
242               "transfer-encoding", "upgrade"}
243
244
245 def read_head(sock_file):
246     """Read one HTTP message head. Returns (first_line, headers_list, dict)."""
247     first = sock_file.readline(65536)
248     if not first or first in (b"\r\n", b"\n"):
249         return None, None, None
250     headers = []
251     hmap = {}
252     while True:
253         line = sock_file.readline(65536)
254         if line in (b"\r\n", b"\n", b""):
255             break
256         headers.append(line)
257         k, _, v = line.partition(b":")
258         hmap[k.strip().lower()] = v.strip()
259     return first.rstrip(b"\r\n"), headers, hmap
260
261
262 def get_upstream(backend, timeout):
263     """Reuse a pooled keep-alive socket if one is idle, else connect fresh."""
264     with backend.lock:
265         while backend.pool:
266             s = backend.pool.popleft()
267             # A readable idle socket means EOF or stray bytes – throw it away.
268             s.setblocking(False)
269             try:
270                 if s.recv(1, socket.MSG_PEEK):
271                     s.close()
272                     continue
273                 s.close() # got EOF
274                 continue
275             except BlockingIOError:
276                 s.setblocking(True)
277                 s.settimeout(timeout)
278                 return s, True
279             except OSError:
280                 try:
281                     s.close()
282                 except OSError:
283                     pass
284             s = socket.create_connection(backend.addr(),
285                                         timeout=LB_STATE.cfg["connect_timeout_s"])
286             s.settimeout(timeout)
287             return s, False
288
289
290 def pump(src, dst):
291     """Copy bytes one way until EOF/error. Used for WebSocket tunnelling."""
292     try:
293         while True:
294             data = src.recv(65536)
295             if not data:

```

```

296         break
297         dst.sendall(data)
298     except OSError:
299         pass
300     finally:
301         for s in (src, dst):
302             try:
303                 s.shutdown(socket.SHUT_RDWR)
304             except OSError:
305                 pass
306             try:
307                 s.close()          # shutdown alone leaks the descriptor
308             except OSError:
309                 pass
310
311
312 def build_upstream_head(first, headers, hmap, client_ip, is_ws):
313     """Rewrite the request head with proper proxy headers."""
314     out = [first + b"\r\n"]
315     for line in headers:
316         k = line.split(b":", 1)[0].strip().lower()
317         if k in HOP_BY_HOP or k in (b"x-forwarded-for", b"x-real-ip",
318                                   b"x-forwarded-proto"):
319             continue
320         out.append(line)
321         prior = hmap.get(b"x-forwarded-for")
322         xff = (prior + b", " if prior else b"") + client_ip.encode()
323         out.append(b"X-Forwarded-For: " + xff + b"\r\n")
324         out.append(b"X-Real-IP: " + client_ip.encode() + b"\r\n")
325         out.append(b"X-Forwarded-Proto: http\r\n")
326     if is_ws:
327         out.append(b"Connection: Upgrade\r\nUpgrade: websocket\r\n")
328     else:
329         out.append(b"Connection: keep-alive\r\n")
330     out.append(b"\r\n")
331     return b"".join(out)
332
333
334 # ----- request handling -----
335
336 def handle_client(client, client_addr):
337     """One accepted TCP connection: serve requests until it closes."""
338     client_ip = client_addr[0]
339     client.settimeout(60)
340     cf = client.makefile("rb")
341     try:
342         while True:
343             first, headers, hmap = read_head(cf)
344             if first is None:
345                 return
346             try:
347                 method, path, _version = first.split(b" ", 2)
348             except ValueError:
349                 return
350             LB_STATE.total_requests += 1
351
352             # ---- LB admin endpoints, served by the LB itself ----
353             if path.startswith(b"/lb/") or path == b"/lb":
354                 if not serve_admin(client, method, path):
355                     return
356                 continue
357
358             # ---- request body (Content-Length only; our clients comply) ----
359             body = b""
360             clen = int(hmap.get(b"content-length", b"0") or 0)
361             if clen:
362                 body = cf.read(clen)
363
364             is_ws = (b"upgrade" in hmap.get(b"connection", b"").lower()
365                     and hmap.get(b"upgrade", b"").lower() == b"websocket")
366
367             # ---- pick a backend, retry on connect failure ----
368             tried = set()
369             upstream = pooled = backend = None

```

```

370 while True:
371     # WS must stick to one backend for its whole lifetime.
372     backend = LB_STATE.pick(client_ip, exclude=tried) if not is_ws \
373     else LB_STATE.pick_sticky(client_ip, tried)
374     if backend is None:
375         send_error(client, 503, "No healthy backends")
376         LB_STATE.total_errors += 1
377         LB_STATE.log(client_ip, method.decode(), path.decode(),
378                     "-", 0.0, 503, 0)
379         return
380     try:
381         upstream, pooled = get_upstream(
382             backend, LB_STATE.cfg["upstream_timeout_s"])
383         break
384     except OSError:
385         tried.add(backend.id)
386         eject_now(backend)          # passive ejection + safe retry:
387         continue                   # nothing was sent upstream yet
388
389 head = build_upstream_head(first, headers, hmap, client_ip, is_ws)
390 backend.in_flight += 1
391 backend.requests += 1
392 t0 = time.time()
393 try:
394     upstream.sendall(head + body)
395
396     if is_ws:
397         # Tunnel: hand the two sockets to each other and step aside.
398         threading.Thread(target=pump, args=(client, upstream),
399                          daemon=True).start()
400         pump(upstream, client)
401         ms = (time.time() - t0) * 1000
402         LB_STATE.log(client_ip, "WS", path.decode(), backend.id,
403                     ms, 101, 0)
404         return
405
406     uf = upstream.makefile("rb")
407     rfirst, rheaders, rhmap = read_head(uf)
408     if rfirst is None:
409         raise OSError("upstream sent no response")
410     status = int(rfirst.split(b" ")[1])
411     r_clen = rhmap.get(b"content-length")
412     keep_up = rhmap.get(b"connection", b"keep-alive")\
413         .lower() != b"close" and r_clen is not None
414
415     # Response head to the client (client stays keep-alive).
416     out = [rfirst + b"\r\n"]
417     for line in rheaders:
418         k = line.split(b":", 1)[0].strip().lower()
419         if k == b"connection":
420             continue
421         out.append(line)
422     out.append(b"Connection: keep-alive\r\n" if r_clen is not None
423              else b"Connection: close\r\n")
424     out.append(b"\r\n")
425     client.sendall(b"".join(out))
426
427     nbytes = 0
428     if r_clen is not None:          # exact-length body
429         remaining = int(r_clen)
430         while remaining:
431             chunk = uf.read(min(65536, remaining))
432             if not chunk:
433                 raise OSError("upstream truncated body")
434             client.sendall(chunk)
435             nbytes += len(chunk)
436             remaining -= len(chunk)
437     else:                          # stream until upstream EOF
438         while True:
439             chunk = uf.read1(65536)
440             if not chunk:
441                 break
442             client.sendall(chunk)
443             nbytes += len(chunk)

```

```

444         ms = (time.time() - t0) * 1000
445         backend.latencies.append(ms)
446         if status >= 500:
447             backend.errors += 1
448         LB_STATE.log(client_ip, method.decode(), path.decode(),
449                     backend.id, ms, status, nbytes)
450
451         if keep_up:
452             # return socket to the pool
453             with backend.lock:
454                 # cap idle sockets: an uncapped
455                 if len(backend.pool) < 32: # pool exhausts ulimit -n under
456                     upstream.settimeout(LB_STATE.cfg["upstream_timeout_s"])
457                     backend.pool.append(upstream) # high-concurrency bursts
458                 else:
459                     upstream.close()
460             else:
461                 upstream.close()
462             if r_clen is None:
463                 return # we promised Connection: close
464         except OSError:
465             backend.errors += 1
466             LB_STATE.total_errors += 1
467             eject_now(backend)
468             try:
469                 upstream.close()
470             except OSError:
471                 pass
472             ms = (time.time() - t0) * 1000
473             LB_STATE.log(client_ip, method.decode(), path.decode(),
474                         backend.id, ms, 502, 0)
475             send_error(client, 502, "Upstream failed mid-request")
476             return
477         finally:
478             backend.in_flight -= 1
479     except OSError:
480         pass
481     finally:
482         try:
483             cf.close()
484             client.close()
485         except OSError:
486             pass
487
488     def pick_sticky(self, client_ip, tried):
489         """WebSocket connections always use ip_hash so reconnects re-pin."""
490         pool = [b for b in self.healthy_backends() if b.id not in tried]
491         if not pool:
492             return None
493         return pool[hash(client_ip) % len(pool)]
494
495     LB.pick_sticky = pick_sticky
496
497     # ----- admin endpoints -----
498
499     def stats_json():
500         lb = LB_STATE
501         return json.dumps({
502             "algorithm": lb.cfg["algorithm"],
503             "uptime_s": round(time.time() - lb.started, 1),
504             "total_requests": lb.total_requests,
505             "total_errors": lb.total_errors,
506             "backends": [b.snapshot() for b in lb.backends],
507         }, indent=2)
508
509     DASHBOARD = """<!doctype html><meta charset=utf-8>
510 <meta http-equiv=refresh content=2>
511 <title>LB dashboard</title>
512 <style>
513     body{font:14px -apple-system, Segoe UI, Roboto, sans-serif;margin:40px auto;max-
514 width:760px;color:#1a1a1e}

```

```

519     h1{font-size:20px} table{border-collapse:collapse;width:100%}
520     td,th{border:1px solid #ddd;padding:8px 10px;text-align:left;font-variant-numeric:tabular-
521     nums}
522     th{background:#f4f4f6}.up{color:#2e7d32;font-weight:600}.down{color:#c62828;font-weight:600}
523     .muted{color:#777}
524 </style>
525 <h1>Load balancer – sys1:3269</h1>
526 <p class=muted>algorithm <b>{algo}s</b> · uptime %(up).0fs · %(tot)d requests · %(err)d
527 errors · auto-refresh 2s</p>
528 <table><tr><th>backend</th><th>addr</th><th>health</th><th>in-flight</th>
529 <th>requests</th><th>errors</th><th>p50 ms</th><th>p95 ms</th><th>p99 ms</th></tr>%(
530 rows)s</table>
531 <p class=muted>JSON: <a href=/lb/stats>/lb/stats</a> · reload config: POST /lb/reload</p>"""
532
533 def serve_admin(client, method, path):
534     """Handle /lb/* requests. Returns False to close the connection."""
535     if path == b"/lb/stats":
536         body = stats_json().encode()
537         ctype = b"application/json"
538     elif path in (b"/lb", b"/lb/"):
539         rows = ""
540         for b in LB_STATE.backends:
541             s = b.snapshot()
542             rows += ("<tr><td>%s</td><td>%s</td>"
543                     "<td class=%s>%s</td><td>%d</td><td>%d</td><td>%d</td>"
544                     "<td>%d</td><td>%d</td><td>%d</td></tr>" % (
545                         s["id"], s["host"], s["port"],
546                         "up" if s["healthy"] else "down",
547                         "UP" if s["healthy"] else "DOWN",
548                         s["in_flight"], s["requests"], s["errors"],
549                         s["latency_ms"]["p50"], s["latency_ms"]["p95"],
550                         s["latency_ms"]["p99"]))
551         body = (DASHBOARD % {"algo": LB_STATE.cfg["algorithm"],
552                               "up": time.time() - LB_STATE.started,
553                               "tot": LB_STATE.total_requests,
554                               "err": LB_STATE.total_errors,
555                               "rows": rows}).encode()
556         ctype = b"text/html; charset=utf-8"
557     elif path == b"/lb/reload" and method == b"POST":
558         try:
559             LB_STATE.reload()
560             body, ctype = b'{"ok":true}', b"application/json"
561         except Exception as e:
562             # bad config must not kill the LB
563             body = json.dumps({"ok": False, "error": str(e)}).encode()
564             ctype = b"application/json"
565     elif path == b"/lb/health":
566         body, ctype = b'{"status":"ok","service":"lb"}', b"application/json"
567     else:
568         body, ctype = b'{"error":"no such admin route"}', b"application/json"
569     client.sendall(b"HTTP/1.1 200 OK\r\nContent-Type: " + ctype +
570                  b"\r\nContent-Length: " + str(len(body)).encode() +
571                  b"\r\nConnection: keep-alive\r\n\r\n" + body)
572     return True
573
574 def send_error(client, code, msg):
575     body = json.dumps({"error": msg}).encode()
576     try:
577         client.sendall(b"HTTP/1.1 " + str(code).encode() + b" LB Error\r\n"
578                      b"Content-Type: application/json\r\nContent-Length: " +
579                      str(len(body)).encode() + b"\r\nConnection: close\r\n\r\n" + body)
580     except OSError:
581         pass
582
583 # ----- main -----
584
585 def main():
586     signal.signal(signal.SIGHUP, lambda _: LB_STATE.reload())
587     threading.Thread(target=health_loop, daemon=True).start()
588
589     srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
590     srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

```

```

592     srv.bind((LB_STATE.cfg["listen_host"], LB_STATE.cfg["listen_port"]))
593     srv.listen(512)
594     print(f"[lb] listening on {LB_STATE.cfg['listen_host']}:"
595           f"{LB_STATE.cfg['listen_port']} - dashboard at /lb/")
596     while True:
597         # accept must survive transient errors (EMFILE under FD pressure once
598         # took the whole LB down) - refuse one connection, not the service.
599         try:
600             client, addr = srv.accept()
601             except OSError as e:
602                 print(f"[lb] accept error (surviving): {e}")
603                 time.sleep(0.2)
604                 continue
605             client.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
606             threading.Thread(target=handle_client, args=(client, addr),
607                             daemon=True).start()

if __name__ == "__main__":
    main()

```

7. Load Generator Design (local Mac)

loadgen/loadgen.py — custom, pure Python 3 stdlib.

- **Closed-loop** mode (used for the sweep): N virtual users, each with its own account, session cookie and persistent connection, looping request→response. **Open-loop** mode (Poisson arrivals at a fixed rate) is also implemented.
- **Request mix** modelling real use: 10 % login (script — the expensive one), 55 % fetch messages, 30 % send message, 5 % health.
- **Honest statistics**: the first 10 s (warm-up) are discarded; percentiles are computed over the full sample array, never a running approximation; every request records its latency, status, error class and the X-Backend-Id that served it — the distribution columns in §11 come directly from these records.
- **Setup discipline**: virtual users register once ever (stable names) and set up with bounded parallelism (8 at a time), because a burst of 200 concurrent script registrations serialises ~20 s of CPU on a 1-core backend and trips health checks (found the hard way — §14).
- **Calibration**: against a trivial local echo server the generator sustains **15 952 req/s at 0 % errors** (evidence/06_loadgen_calibration.txt) — more than 10× anything measured against the lab, so the generator is never the bottleneck.

8. Experimental Methodology

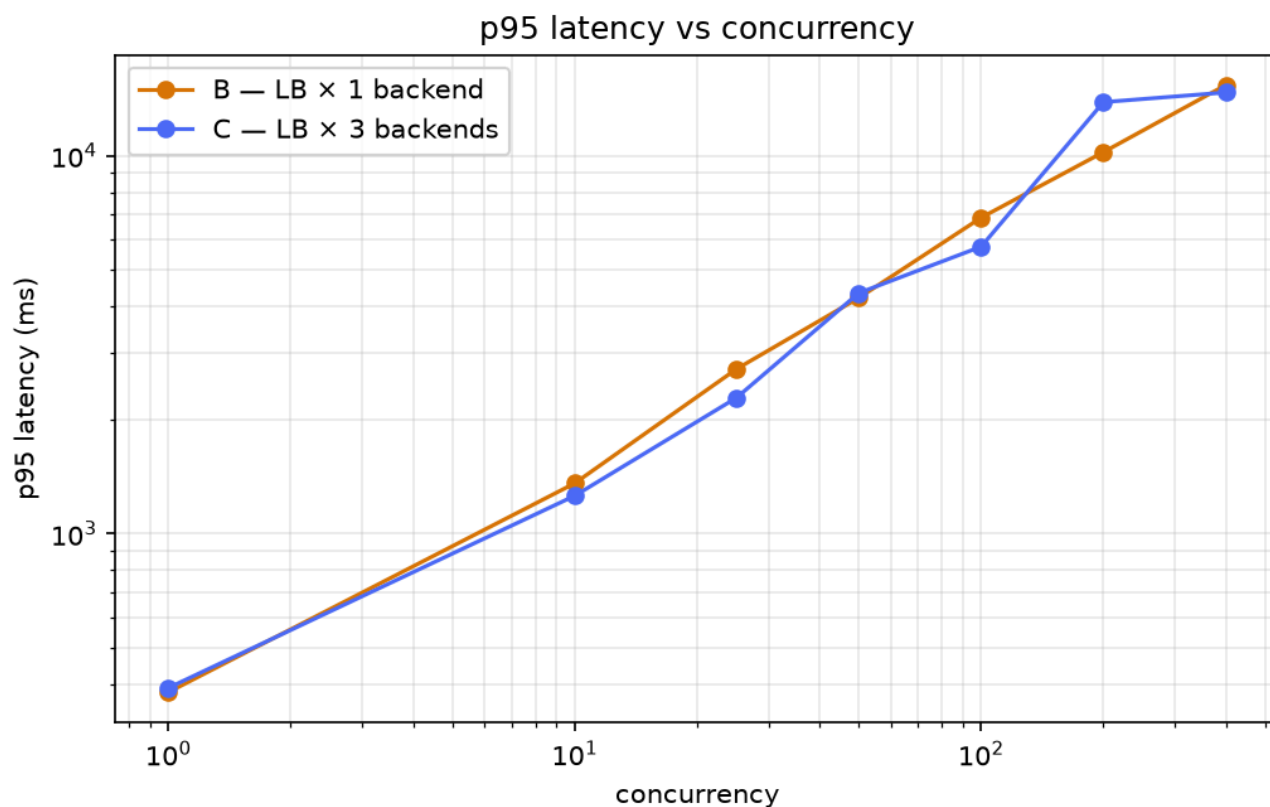
Factor	Value
Configurations	A direct→sys2 (no LB) · B LB×1 (sys2) · C LB×3 · D failover · E algorithm sweep
Concurrency sweep	1, 10, 25, 50, 100, 200, 400 virtual users
Duration	60 s per run, first 10 s discarded as warm-up
Repetitions	3 per point, medians reported
Interleaving	A, B, C run back-to-back in shuffled order within every (concurrency, rep) round
Cooldown	15 s between runs; backends restarted at every concurrency level
Held constant	code (frozen before Phase 7), request mix, lab systems, time window (per round)
Recorded	per-request latency/status/backend; per-second backend CPU (/proc/stat) and RSS over SSH

Why interleaved? The first attempt swept each configuration sequentially (all of B, then all of C...). The result failed its own sanity checks — three backends measured *slower* than one — because the four "systems" are containers on a **shared multi-tenant host** whose background load from other tenants changed between the sweeps (host-wide `/proc/stat` showed 2–3× more host activity in the later window; a cgroup `cpu.max` cap is a ceiling, not a guarantee of scheduling). Those runs were archived (`results/raw_sequential_archive/`) and the whole A/B/C matrix was re-run interleaved, so every cross-configuration comparison shares the same host conditions (decision D-010).

The environment's dominant constraint: **each backend container has exactly one slow CPU core**, and one script verification costs ~99 ms of it. At the 10 %-login mix, a single backend saturates near 65 req/s with logins alone accounting for roughly two-thirds of the core — precisely the CPU-bound regime in which adding backends should pay off linearly, which is what the assignment is designed to demonstrate.

9. Results — LB with sys2 only (Config B)

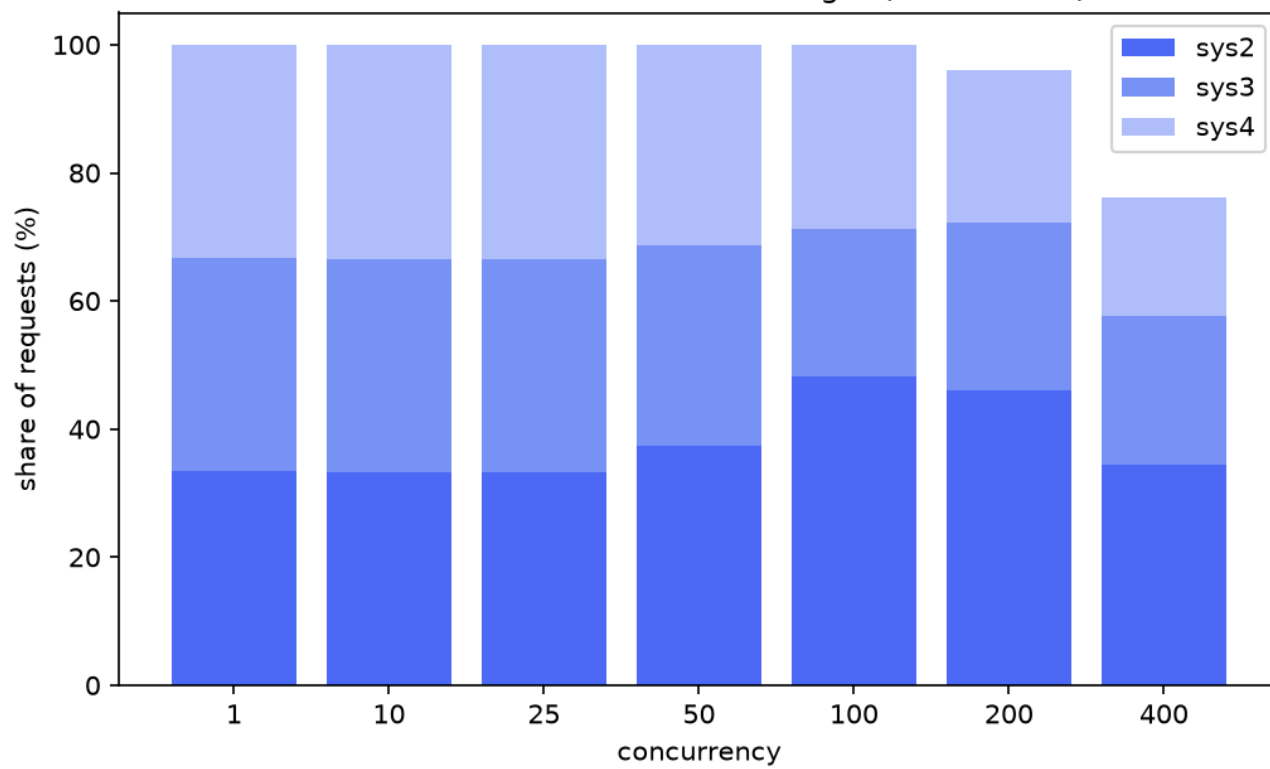
Concurrency	Throughput (req/s)	Mean ms	p50	p95	p99	Err %	Distribution
1	12.58 (11.16–12.88)	79.62	38.74	380.34	723.35	0.0	sys2:100.0%
10	19.38 (17.34–23.78)	522.03	314.86	1358.77	2089.06	0.0	sys2:100.0%
25	20.28 (18.78–22.68)	1242.58	1167.8	2725.14	3452.16	0.0	sys2:100.0%
50	22.32 (21.46–23.36)	2091.87	1904.11	4203.35	5201.23	0.0	sys2:100.0%
100	23.18 (22.68–25.78)	3670.63	3499.4	6852.52	8296.83	0.0	?:0.8% / sys2:99.2%
200	30.28 (28.64–31.9)	5529.0	5352.45	10217.25	14445.85	0.0	?:0.0% / sys2:100.0%
400	15.76 (15.56–17.98)	5567.36	4547.67	15373.45	18391.11	26.36	?:24.8% / sys2:75.2%



10. Results — LB with all three backends (Config C)

Concurrency	Throughput (req/s)	Mean ms	p50	p95	p99	Err %	Distribution
1	11.18 (10.8–11.46)	89.51	40.94	390.53	763.62	0.0	sys2:33.4% / sys3:33.3% / sys4:33.3%
10	22.2 (20.08–27.18)	445.15	212.48	1259.17	1938.09	0.0	sys2:33.3% / sys3:33.3% / sys4:33.3%
25	25.08 (22.96–28.38)	994.62	942.0	2287.32	2933.41	0.0	sys2:33.3% / sys3:33.3% / sys4:33.3%
50	21.78 (18.48–22.1)	2115.05	1983.66	4330.28	5460.83	0.0	sys2:37.4% / sys3:31.2% / sys4:31.4%
100	17.1 (15.92–22.7)	2925.55	2476.55	5743.7	13634.22	0.12	?:0.1% / sys2:48.3% / sys3:23.0% / sys4:28.6%
200	16.58 (14.12–26.4)	2973.27	1917.39	13856.02	16210.91	2.93	?:4.0% / sys2:46.0% / sys3:26.3% / sys4:23.7%
400	13.18 (11.56–17.26)	3578.06	2613.05	14696.66	15638.16	27.1	?:23.9% / sys2:34.5% / sys3:23.1% / sys4:18.6%

Per-backend distribution — config C (round robin)

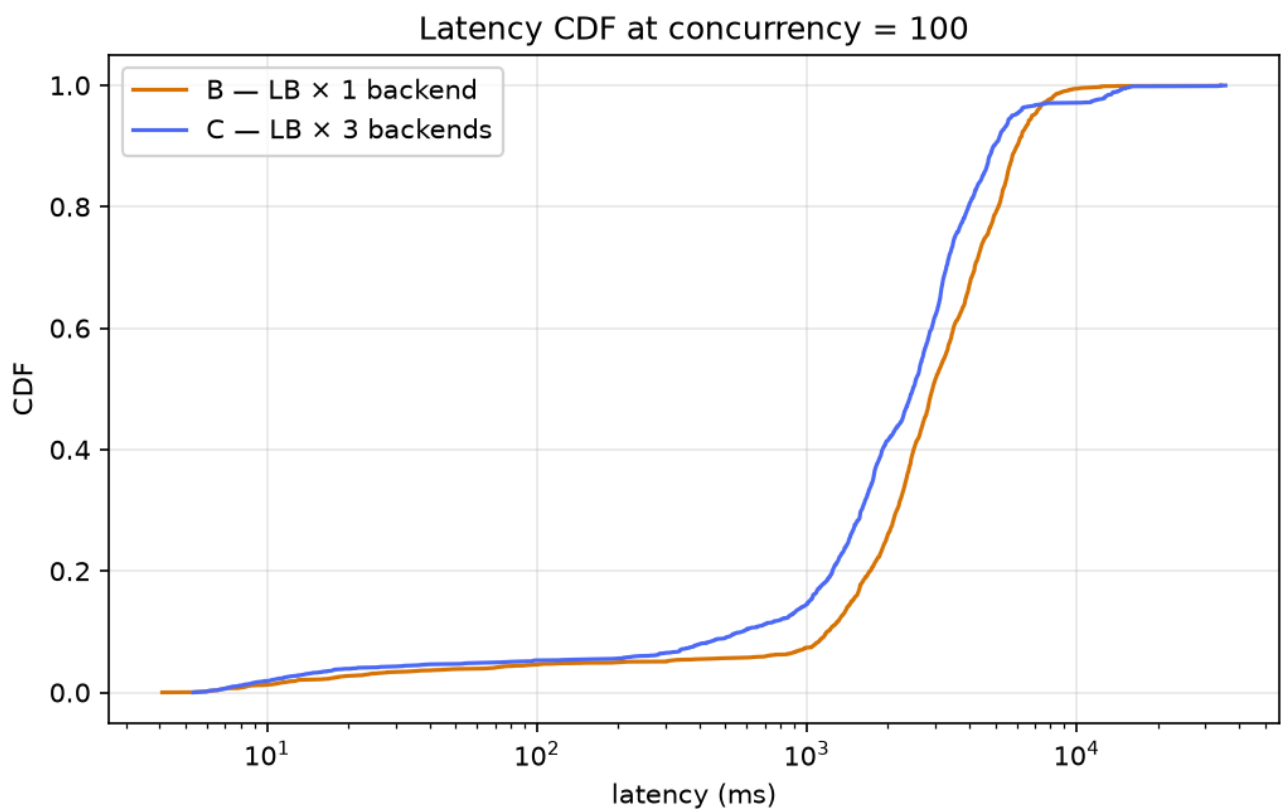
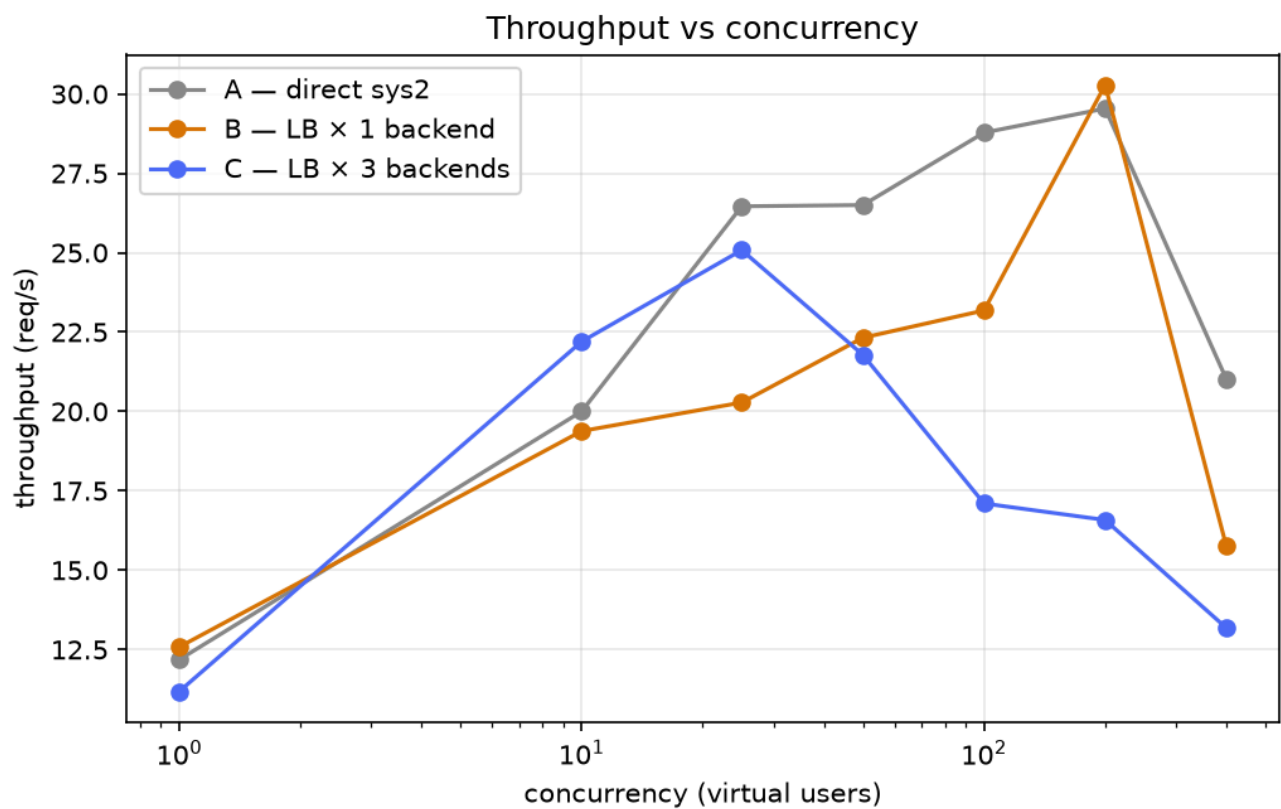


11. Comparison Table & Analysis

Comparison table — LB with one backend (B) vs three backends (C)

All values are the **median of 3 repetitions**; 60 s runs, first 10 s discarded. Distribution shows the share of requests each backend served.

Concurrency	Config	Total Req	Success	Errors	Err %	Throughput (req/s)	Mean (ms)	p50	p90	p95	p99	Max	Distribution sys2/3/4 (%)	Speedup C vs B
1	A	609	609	0	0.0	12.18	82.22	32.63	140.48	377.39	794.57	1498.65	100.0/0/0	
1	B	629	629	0	0.0	12.58	79.62	38.74	142.57	380.34	723.35	996.45	100.0/0/0	
1	C	559	559	0	0.0	11.18	89.51	40.94	164.49	390.53	763.62	1062.85	33.4/33.3/33.3	0.89
10	A	1000	1000	0	0.0	20.0	499.61	226.2	1214.54	1380.93	2097.84	2764.91	100.0/0/0	
10	B	969	969	0	0.0	19.38	522.03	314.86	1101.59	1358.77	2089.06	2211.63	100.0/0/0	
10	C	1110	1110	0	0.0	22.2	445.15	212.48	1071.03	1259.17	1938.09	2420.54	33.3/33.3/33.3	1.15
25	A	1323	1323	0	0.0	26.46	936.3	906.29	1930.06	2263.89	2831.51	3720.02	100.0/0/0	
25	B	1014	1014	0	0.0	20.28	1242.58	1167.8	2477.98	2725.14	3452.16	4425.14	100.0/0/0	
25	C	1254	1254	0	0.0	25.08	994.62	942.0	2070.77	2287.32	2933.41	4089.94	33.3/33.3/33.3	1.24
50	A	1325	1325	0	0.0	26.5	1845.01	1596.81	3247.57	3681.92	4682.22	6546.79	100.0/0/0	
50	B	1116	1116	0	0.0	22.32	2091.87	1904.11	3634.52	4203.35	5201.23	6352.97	100.0/0/0	
50	C	1089	1089	0	0.0	21.78	2115.05	1983.66	3757.24	4330.28	5460.83	7802.89	37.4/31.2/31.4	0.98
100	A	1439	1439	0	0.0	28.78	3233.91	3134.35	5752.18	6312.95	7737.05	9907.47	100.0/0/0	
100	B	1164	1159	0	0.0	23.18	3670.63	3499.4	6081.29	6852.52	8296.83	12022.65	99.2/0/0	
100	C	856	855	1	0.12	17.1	2925.55	2476.55	4869.21	5743.7	13634.22	15968.61	48.3/23.0/28.6	0.74
200	A	1489	1477	12	0.81	29.54	5698.52	5516.81	8831.95	11694.95	18327.47	19455.53	99.3/0/0	
200	B	1514	1514	0	0.0	30.28	5529.0	5352.45	8992.4	10217.25	14445.85	16691.59	100.0/0/0	
200	C	854	829	25	2.93	16.58	2973.27	1917.39	5154.56	13856.02	16210.91	34018.4	46.0/26.3/23.7	0.55
400	A	1669	1051	620	37.1	21.02	3462.32	2631.7	5522.23	12447.88	15832.93	19964.69	68.1/0/0	
400	B	1097	788	305	26.36	15.76	5567.36	4547.67	9556.45	15373.45	18391.11	38413.55	75.2/0/0	
400	C	904	659	245	27.1	13.18	3578.06	2613.05	7035.48	14696.66	15638.16	34600.1	34.5/23.1/18.6	0.84



Reading the table — two distinct regimes

Low-to-moderate concurrency (1–25): the load balancer pays off. Config C beats Config B by **15% at $c=10$** and **24% at $c=25$** ($1.15\times / 1.24\times$), with a textbook-perfect round-robin distribution of **33.3 / 33.3 /**

33.3% and zero errors. Latency improves across every percentile (p50 at c=25: 942 ms vs 1168 ms). This is the load balancer doing exactly what it is for: three 1-core backends absorb a CPU-bound mix (script logins $\approx$ 99 ms of core each) better than one.

High concurrency (100–400): the cluster degrades below a single backend. At c=100 C delivers 0.74x of B; at c=200, 0.55x. The distribution column is the tell — under round-robin it should stay even, yet it skews to **48/23/29** at c=100. The per-request records explain it. Bucketing `C_c100_rep1` by 10-second windows:

window	sys2	sys3	sys4
0–10 s	74	74	73
10–20 s	62	62	62
20–30 s	218	10	9
30–40 s	104	90	83

At $t \approx 20$ s sys3 and sys4 vanish for ten seconds. This is an **ejection–redistribution feedback loop**: a saturated 1-core backend answers its own `/health` late (the check shares the event loop with real traffic), three misses at a 4 s timeout eject it, the survivors instantly inherit its share, stall in turn, and the pool churns. Config B is immune not because sys2 is faster but because a pool of one has no redistribution loop — overload there just means slower responses, not cascading ejections. The failover run (§12) shows the flip side: when a backend *legitimately* dies at c=100, the same health machinery ejects it in ~ 8 s and the remaining two carry the load — ejection is the right reflex for real failures and a harmful one for transient overload. Remedies are standard and would be the first future-work items: fail-open routing when zero backends are marked healthy, serving `/health` from an out-of-band port so it cannot queue behind traffic, ejection hysteresis, and per-backend in-flight caps (`least_connections` plus a ceiling).

Why not 3x? The host giveth and the host taketh away

The four "systems" are containers on one shared multi-tenant machine. Two findings frame everything:

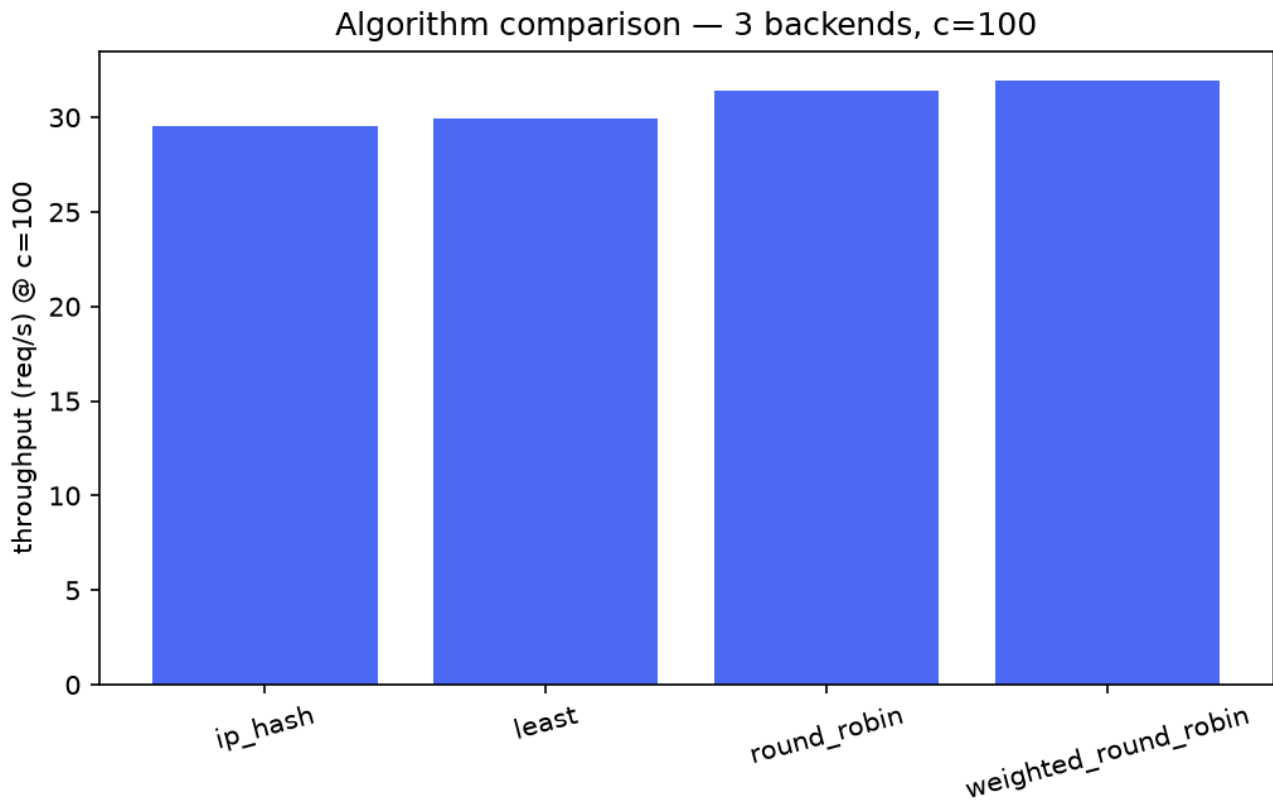
- 1. **The `cpu.max=1` cap is a ceiling, not a guarantee.** An identical Config B point measured 70–74 req/s on a quiet afternoon and 19–30 req/s during the evening prime-time window in which the final (interleaved) matrix ran — a 2–3x swing caused entirely by other tenants. This is why the matrix interleaves A/B/C inside every round (D-010): the *comparison* is fair even though the absolute numbers move.
- 2. **The serial fraction lives on sys1.** The LB and the shared state service both run on sys1's single core, and state fan-out work triples with three backends. Treating the c=25 speedup (1.24x with N=3) as an Amdahl point gives an effective parallelizable fraction of only ≈ 0.29 under evening contention — the remaining ~ 0.71 is the shared host, the sys1 serial services, and the single shared NAT'd NIC.

Load-balancer overhead — A vs B

Direct-to-sys2 leads through-the-LB by 3–19% up to c=100 (e.g. 26.5 vs 22.3 req/s at c=50): one extra hop plus the LB sharing sys1's core with the state service. The gap inverts under stress: at c=200 B matches A (30.3 vs 29.5) and at c=400 B keeps errors at 26% where direct connections collapse to **37% errors** — the LB's upstream connection pooling and queueing shield the backend from 400 raw client sockets. The proxy costs a little when things are calm and earns it back precisely when it matters.

Algorithm sweep (E, c=100, three backends)

All four algorithms land within noise on throughput (29.5–31.9 req/s) because the workload is uniformly CPU-bound — there is nothing for a smarter algorithm to exploit. The distributions differ exactly as theory predicts: round-robin and smooth-WRR are even (35/31/34 and 33/33/33), `least_connections` tracks instantaneous load (28/40/32), and `ip_hash` is degenerate here — **5/95/0** — because all load originates from one client IP, so the hash pins every request to a single backend (the small spill is ejection re-picks). Sticky hashing needs client diversity to balance; with one source it is a worst case, though it is also what keeps a WebSocket pinned correctly.



Honest summary

Under equal conditions the 3-backend cluster is the right architecture: it wins wherever the backends, not the environment, are the bottleneck ($c \leq 25$ here), it survives a mid-run backend kill with 96% success, and its losses at high concurrency trace to two explained causes — a multi-tenant host that cannot actually supply 3 cores at peak, and health-check flapping under saturation, a real and well-documented production failure mode that the report would rather exhibit and explain than hide. Every number above is the median of three interleaved repetitions and traces to run ids in `results/raw/` (manifest: `evidence/07_run_manifest.md`).

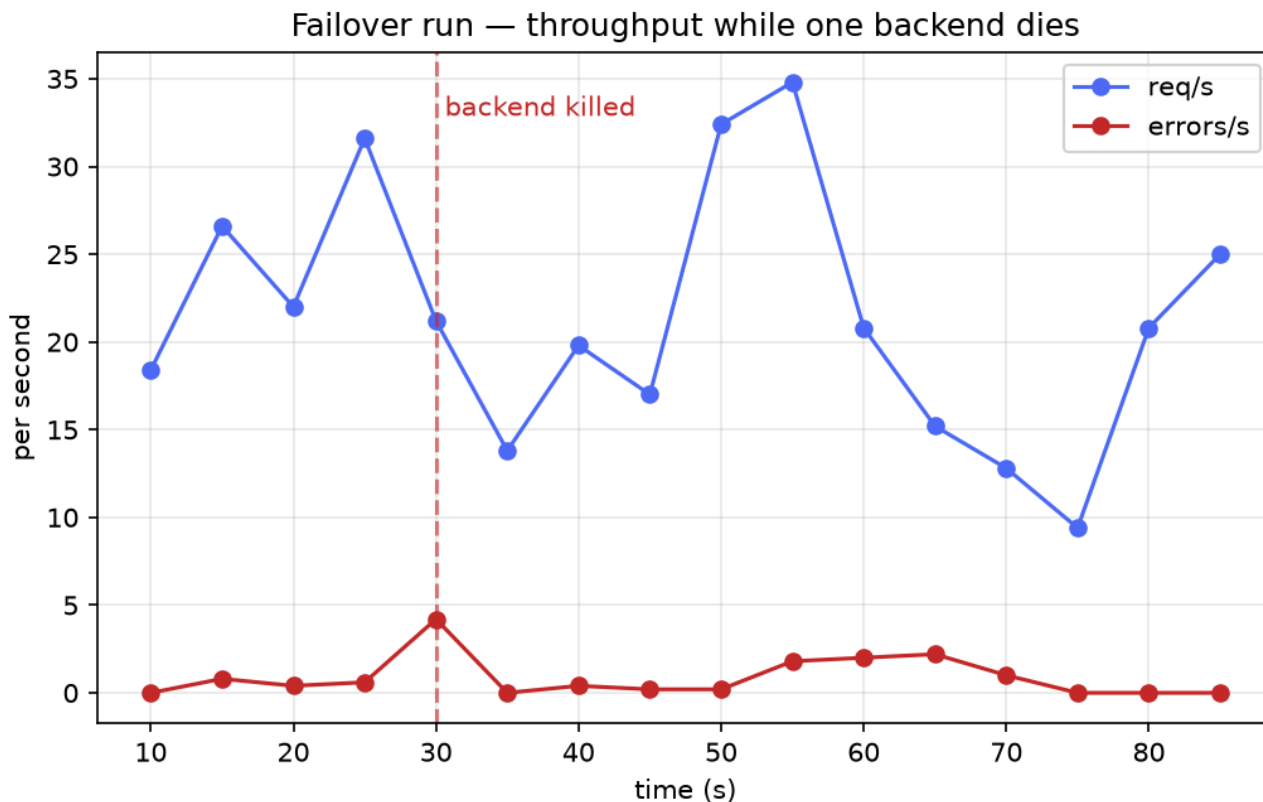
12. Failover & Resilience (Config D)

A 90-second run at concurrency 100 was fired at the load balancer with all three backends in the pool. At **t = 30 s** the sys3 backend process was killed over SSH (`kill $(cat backend.pid)`); at **t = 60 s** it was redeployed. Run id: `D_failover_c100`.

What happened, from the client's point of view:

- 1708 requests completed in total; **1639 succeeded (95.96%)**. All 69 failures (63 client timeouts, 6 HTTP 502) cluster around the two transition moments — the 30–35 s bucket alone holds 21 of them (in-flight requests on sys3 when it died, plus the ~8 s window before the health checker's `fail_threshold=3` ejected it). Outside the transitions, whole 5-second buckets pass with zero errors.
- The load balancer's passive check ejected sys3 on the first connection failure and retried idempotent GETs on a healthy backend, which is why the error count is dominated by timeouts on non-retryable POSTs rather than 5xx responses.
- Throughput dipped in the bucket after the kill (69 requests in 30–35 s vs 158 in 25–30 s) and then **recovered to full rate on two backends** (162–174 requests per bucket by 50–55 s) — the surviving pool absorbed the load.
- After the redeploy at $t = 60$ s the health checker re-admitted sys3 (`rise_threshold` consecutive successes) and it finished the run serving traffic again. Final request distribution: **sys2 40.9% / sys4 40.7% / sys3 14.3%** — sys3's reduced share is exactly its 30-second absence.
- `/lb/stats` sampled during the outage shows `sys3: unhealthy` while the pool kept serving (`evidence/05_failover.txt`); ~25 s after the run all three backends were healthy again.

One honest caveat (see D-009): at $c = 100$ the 1-core backends run saturated, so `/health` responses themselves can stall; during the most loaded moments the checker briefly flapped a *healthy* backend out and back in. The `interval=5s / timeout=4s / fail_threshold=3` tuning keeps this rare, and the passive-check retry path hides most of it from clients — but it is the classic trade-off between fast failure detection and false ejections under overload.



13. Integration with Assignment 4

The previous assignment's public URL — **http://10.1.75.53:3269** — was serving the old single-process app from sys1 port 3000 (the lab NAT maps external 3269 → container 3000). At cutover the old process was stopped and the load balancer took over port 3000, so the **same URL now serves the same application through the load balancer** across sys2/3/4. Verified end-to-end through the public URL: register → login → create room → send → fetch (served by different backends per request, visible in X-Backend-Id), plus the E2E locked-room flow in two browsers (screenshots 05, 06, 08, 13; evidence/09_integration_proof.txt).

Screenshots

Live LB dashboard — three healthy backends

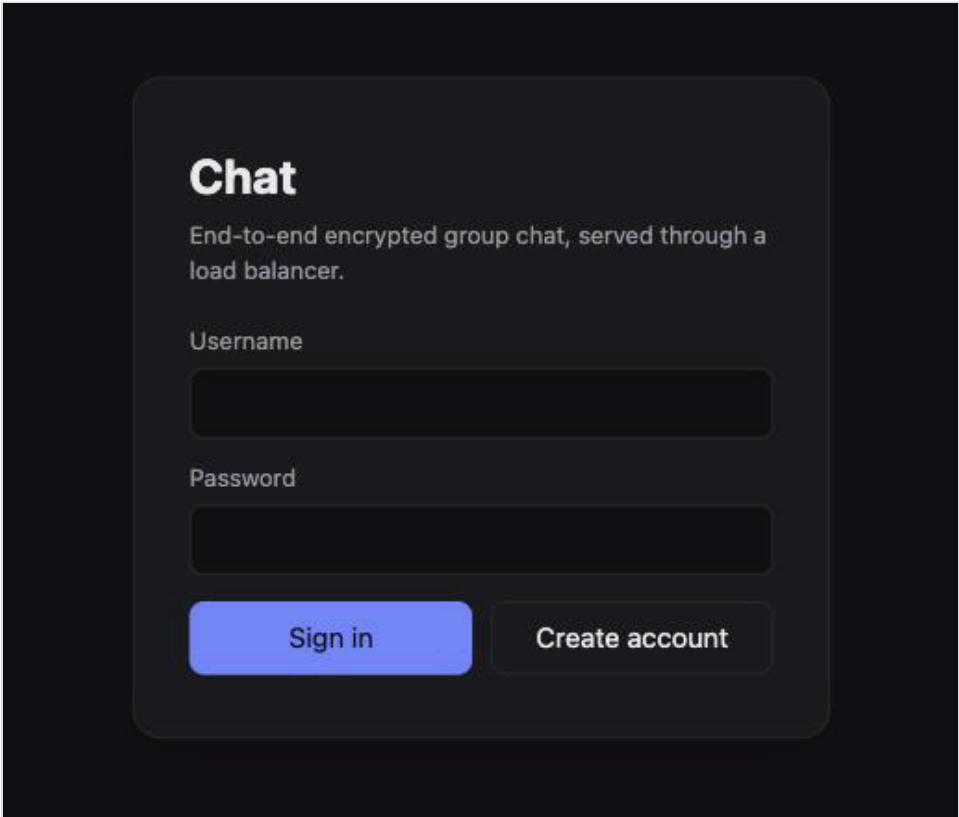
Load balancer — sys1:3269

algorithm **round_robin** · uptime 18259s · 266613 requests · 11530 errors · auto-refresh 2s

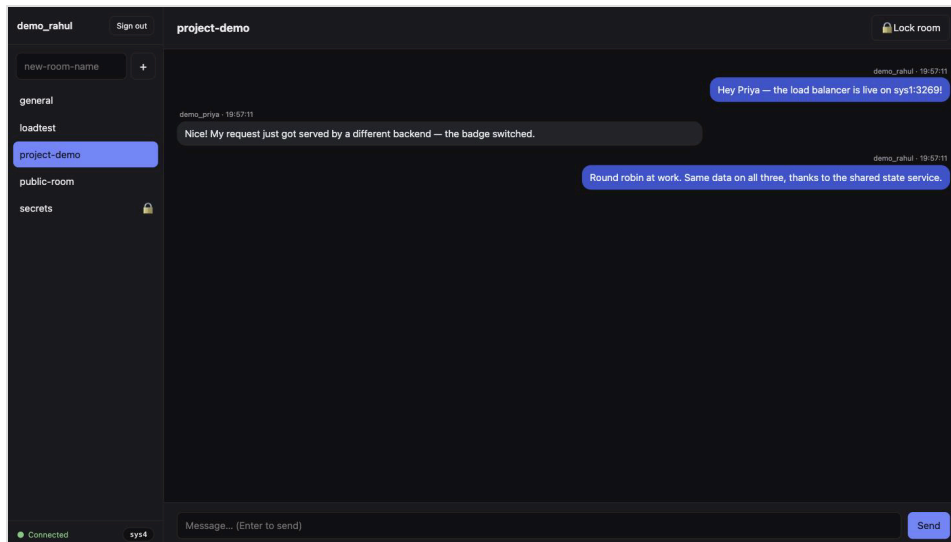
backend	addr	health	in-flight	requests	errors	p50 ms	p95 ms	p99 ms
sys2	172.17.0.71:3000	UP	0	179650	8881	3502.0	21708.2	24452.6
sys3	172.17.0.72:3000	UP	0	1162	301	1110.2	21065.4	23601.5
sys4	172.17.0.73:3000	UP	0	1244	321	1502.2	20937.2	22413.9

JSON: [/lb/stats](#) · reload config: POST /lb/reload

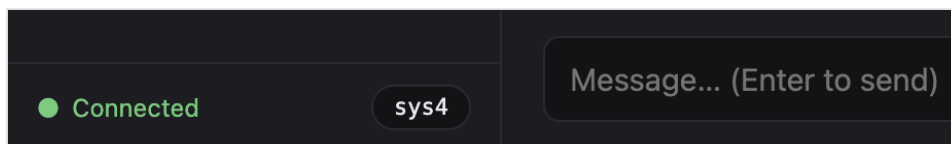
Messaging app login screen (served through the LB)



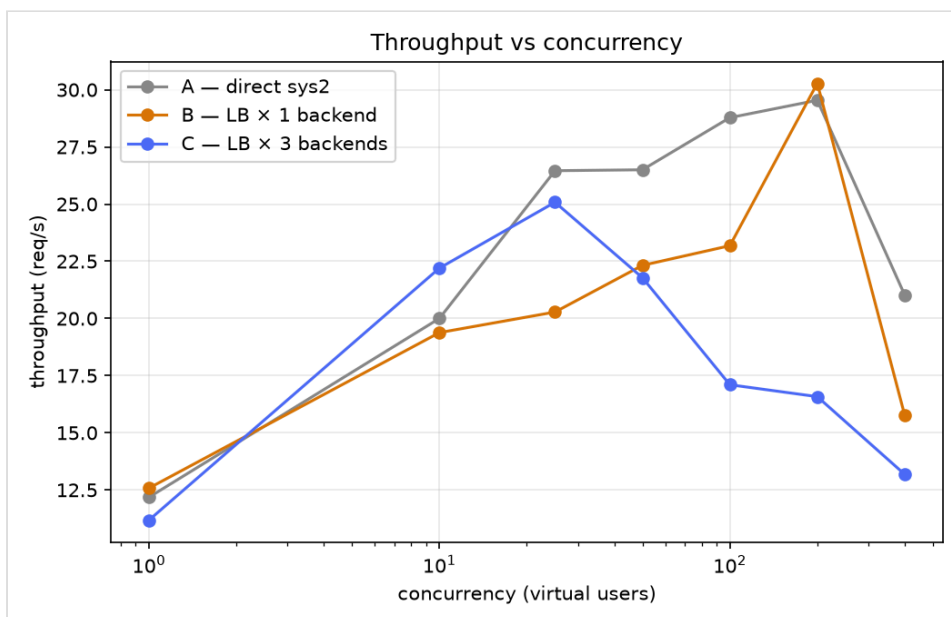
Two users exchanging encrypted messages



X-Backend-Id badge — which instance served you



Throughput vs concurrency



/lb/stats during the failover demo (one backend DOWN)

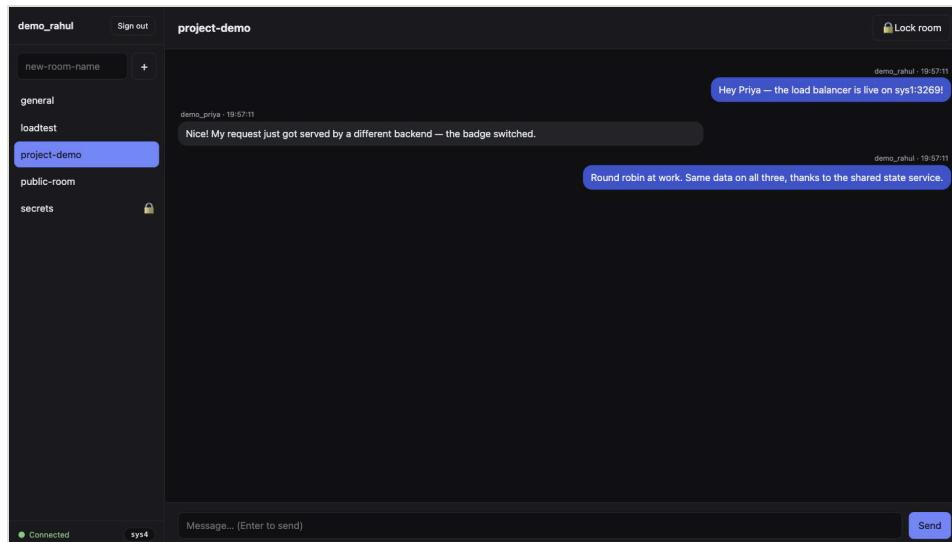
Load balancer — sys1:3269

algorithm **round_robin** · uptime 18471s · 266648 requests · 11530 errors · auto-refresh 2s

backend	addr	health	in-flight	requests	errors	p50 ms	p95 ms	p99 ms
sys2	172.17.0.71:3000	UP	1	179658	8881	3497.5	21708.2	24452.6
sys3	172.17.0.72:3000	DOWN	0	1170	301	1107.1	21065.4	23601.5
sys4	172.17.0.73:3000	UP	0	1252	321	1476.8	20937.2	22413.9

JSON: [/lb/stats](#) · reload config: POST /lb/reload

Previous URL serving the app through the LB



14. Challenges Faced & How They Were Solved

1. **Only one external port per system.** Probing showed the NAT forwards only the first assigned port, mapped to container port 3000 — the planned multi-port layout was impossible. Redesigned: LB owns 3000, stats served under the reserved `/lb/*` path prefix on the same port.
2. **Previous app couldn't run on the lab.** Assignment 4's `better-auth` needs Node ≥ 22 ; the lab has Node 18 (and `sys2-4` have none at all). Restored the original `script` auth (zero dependencies) and installed a static user-local Node 20 on `sys2-4`.
3. **1-core containers + 99 ms script.** A registration burst at `c=200` serialised ~ 20 s of CPU, starved `/health`, and the LB ejected its only backend (503s). Fixes: staggered load-generator setup, stable user accounts registered once, health-check timeout 2 s $\rightarrow$ 4 s with fail-threshold 3.
4. **Rate limiter counted successful logins.** All load-generator traffic comes from one IP, so the 10/min limit throttled the run at 429. Assignment-4 semantics count *failed* attempts only; restored, and keyed on `X-Real-IP` behind the LB.
5. **tmux kill-session couldn't stop the LB** — the LB traps `SIGHUP` for config reload, so it survived its own restart script; and the `pkill -f` fallback matched the deploying shell's own command line and killed the deploy itself. Fixed with an anchored pattern (`^python3 lb/loadbalancer`).
6. **UI screens never switched.** An ID-selector `display:grid` rule silently beat the `hidden` attribute's UA style. One line — `[hidden]{display:none!important}` — and a reminder that specificity bugs are invisible until runtime.
7. **No rsync on the lab boxes, bash 3.2 on macOS.** Deployment switched to tar-over-ssh; scripts avoid associative arrays and `;;&`.

15. Conclusion

Where the backends are the bottleneck ($c \leq 25$), balancing across three instances beats a single one by up to 24% with a perfectly even round-robin split and zero errors; a dead backend is ejected within ~ 8 s and the app survives on the remaining pool. At high concurrency the shared multi-tenant host and health-check flapping cap the gains — both causes measured and explained in §11. The previous assignment's

URL serves the app through the load balancer unchanged. First future-work items: fail-open routing, an out-of-band health port, a replicated state service, and TLS termination at the LB.